Markdown
技術文件撰寫

什麼是 Markdown？

Markdown 是一種「輕量級標記語言」。

不同於 Word 需要繁瑣的滑鼠操作，Markdown 透過簡單符號實現結構化排版。文字即內容，符號即設計。

純文字的威力

所有的 .md 檔案本質上都是 .txt。這意味著：

- 檔案體積極小
- 100% 跨平台相容
- 永不過時的格式

渲染流程 (Parsing)

1. 寫入源碼 ➔ 2. 瀏覽器/編輯器解析 ➔ 3. 渲染為具備樣式的 HTML 網頁。

核心應用場景

寫程式作業報告

團隊專題協作

個人知識管理

與 AI 工具對話

場景一：寫程式作業報告

❌ 用 Word 寫報告

代碼貼進去，排版全亂 (空格/縮排失蹤)。

對方電腦開啟後格式全跑掉。

為了排版得截圖，截圖 內容無法複製程式。

✅ **用 Markdown**：

- 程式碼區塊自動語法高亮
- 純文字檔，任何電腦都能開
- 上傳 GitHub 自動顯示漂亮格式

場景二：團隊專題協作

❌ 用 Google Docs 的隱憂

多人同時編輯同一個代碼區塊，容易發生「蓋掉」慘劇。

不知道誰在什麼時候修改了哪一段 說明。

無法明確看到「修改前」與「修改後」的差異 (Diff)。

✅ Git + Markdown 的黃金組合

文件與代碼一起存放在 Repo，同步更新。

每一行的異動都記錄了誰是負責人。

完美支援 Merge 操作，解決版本衝突。

「這是在業界工作的標配工作流程。」

場景三：筆記的長久保存

檔案永不過時

10 年後的電腦可能開不了 Word 2025，但絕對讀得了純文字。Markdown 確保您的心血不會因為軟體更新而消失。

極速搜尋

純文字搜尋速度比二進制檔案 (doc/pdf) 快上百倍。在數千份筆記中秒搜關鍵字。

AI 能理解程式碼、檔案、步驟的邏輯。當您用 MD 語法與 AI 溝通時:

場景四:與 AI 協作(現在的主流)

用不規範的格式

- AI 看不懂您的程式碼在哪
- AI 誤解您的意思
- AI 回答產生幻覺

用 Markdown 標準格式

- AI 準確辨識程式碼、檔案名、步驟
- 減少內文認知的誤解
- 協作效率大幅提升

業界怎麼用

Markdown 在軟體開發中已經是常見的**標準格式**

🏢 軟體大廠的文件標準

- **技術文件**：API 說明、系統架構文件
- **專案管理**：需求文件、會議記錄、專案規劃
- **程式碼註解**：README.md、CHANGELOG.md、CONTRIBUTING.md
- **實際案例**：
  - Google 的 Android 開源專案使用 Markdown 撰寫文件
  - Meta 的 React 官方文件使用 Markdown
  - Microsoft 的 VSCode 專案文件全部用 Markdown

Markdown 的核心優點

✅ 優點

優點	說明	實際好處
學習曲線低	30 分鐘就能上手基本語法	不像 LaTeX 要學好幾天
純文字格式	用記事本就能開	10 年後還能用
檔案超小	幾 KB 而已	Word 檔動輒幾 MB
版本控制友善	Git 可以完美追蹤	團隊協作超方便
跨平台	Windows、Mac、Linux 都能用	不受作業系統限制
AI 友善	結構清晰易解析	減少 AI 幻覺
免費	不需要買 Office	省錢！

❌ 缺點

缺點	說明	解決方法
排版功能有限	不能像 Word 那樣精細調整	對於程式文件來說夠用了
需要預覽工具	不能直接看到效果	Antigravity 內建預覽
表格複雜時難寫	超過 5 欄會很亂	改用程式生成表格
不適合列印	精美排版要轉成 PDF	用工具轉換即可

Markdown vs Word：該用哪個？

使用場景	推薦工具	原因
程式作業報告	✅ Markdown	程式碼排版好
團隊專題文件	✅ Markdown	版本控制方便
技術筆記	✅ Markdown	搜尋快、不會壞
與 AI 協作	✅ Markdown	AI 容易理解
精美海報	❌ Word/PPT	需要複雜排版
正式公文	❌ Word	需要特定格式
履歷	兩者皆可	看行業習慣

第二章:核心語法

建立首份 Markdown 檔案

1. 在檔案總管右鍵點擊資料夾 ➔「新增檔案」
2. 命名為：我的筆記.md (副檔名必為 **.md**)
3. 按 **Ctrl+Shift+V** (Windows) 或 **Cmd+Shift+V** (Mac)
4. 開始您的第一行程式碼說明！

標題語法（Headings）

使用 # 符號，數量代表層級：

```
# 一級標題（最大）

## 二級標題

### 三級標題

#### 四級標題

##### 五級標題

###### 六級標題（最小）
```

- ✅ **一個文件只用一個 `#` 一級標題**（作為文件標題）
- ✅ **按照層級順序使用**（不要跳級，例如從 `##` 直接跳到 `####`）
- ✅ **標題要具體明確**，避免「這個」、「那個」等模糊詞彙
- ❌ 不要寫：`## 關於這個功能`
- ✅ 要寫：`## 使用者登入功能說明`

⚠️ **注意：符號後方必須有一個半角空格。**

段落與換行原則

段落 (Paragraphs)

兩段文字間**必須空一行**。這是 AI 辨識段落邊界的唯一依據。

強制換行

在行尾加**兩個空格**再換行。避免產生不必要的大間距。

段落：兩段文字之間空一行
換行：在行尾加兩個空格，或直接空一行

```
這是第一段。

這是第二段（中間空了一行）。

這是第三段的第一句。
這是第三段的第二句（上一行結尾有兩個空格）。
```

- ✅ **段落之間一定要空一行**，讓 AI 清楚辨識段落邊界
- ✅ **避免超長段落**，建議一段不超過 5-7 行

無序列表（Unordered Lists）

語法（無序列表使用 `-`、`*` 或 `+`）

```
- 第一項
- 第二項
  - 子項目（縮排）
- 第三項
```

顯示效果

- 第一項
- 第二項
  - 子項目
- 第三項

有序列表（Ordered Lists）

語法（數字加點）

```
1. 第一步
2. 第二步
3. 第三步
   1. 子步驟（縮排）
```

渲染邏輯

1. 第一步
2. 第二步
3. 第三步
   1. 子步驟

- ✅ **統一使用 `-` 作為無序列表符號**（不要混用 `-`、`*`、`+`）
- ✅ **子列表要正確縮排**（2 或 4 個空格）
- ✅ **列表項目要簡潔明確**
- ❌ 不要寫：`- 這個東西很重要`
- ✅ 要寫：`- 使用者密碼必須包含 8 個字元以上`

文字修飾與強調

```
**粗體文字**
_斜體文字_
**_粗斜體_**
~~刪除線~~
`程式碼或指令`
```

顯示效果：

粗體文字

斜體文字

粗斜體

~~刪除線~~

`程式碼或指令`

💡 重點概念用**粗體**，技術術語用行內代碼。

- ✅ **用反引號 `` ` `` 標註程式碼、檔案名、指令**
  - 範例：請開啟 `index.tsx` 檔案
  - 範例：執行 `npm install` 指令
- ✅ **用粗體強調重要概念**
- ✅ **避免過度使用格式**（不要每個字都加粗體）

超連結 [顯示文字](網址)

連結

```
[顯示文字](https://網址)
[Google](https://www.google.com)
```

顯示效果：Google

圖片

```
![圖片說明文字](圖片網址或路徑)
![範例圖片](https://example.com/image.png)
```

- ✅ **連結文字要具體**，讓 AI 知道連結的內容
  - ❌ 不要寫：`[點這裡](URL)`
  - ✅ 要寫：`[GitHub 官方文件](URL)`
- ✅ **圖片說明要清楚描述圖片內容**
  - ❌ 不要寫：`![圖片](path)`
  - ✅ 要寫：`![使用者登入流程圖](path)`
- ✅ **檔案路徑使用絕對路徑或明確的相對路徑**

多行程式碼區塊（Code Blocks）

行內程式碼

```
執行 `git commit` 指令
```

多行程式碼區塊

務必標註語言名稱（如 javascript, python）！

```
```javascript
function hello() {
  console.log("Hello, World!");
}
```
```

顯示效果：

```javascript
function hello() {
  console.log("Hello, World!");
}
```

- ✅ **一定要標註程式語言**（例如 `javascript`、`python`、`bash`）
  - ❌ 不要寫：```（沒有語言標註）
  - ✅ 要寫：```javascript 或 ```python
- ✅ **程式碼要有適當的註解**
- ✅ **避免在程式碼區塊中使用「...」省略**
  - ❌ 不要寫：

    ```
    function example() {
      // ... 其他程式碼
    }
    ```

  - ✅ 要寫完整的程式碼，或明確說明省略的內容

為什麼標註語言標籤極其重要？

視覺高亮

自動著色，讓代碼邏輯更易 閱讀。

AI 識別

告訴 AI 該代碼的語法特徵，避免誤判。

專業感

業界標準 README 必備配置。

引用區塊 (Blockquotes)

使用 > 符號開頭:

```
> 這是引用文字
> 可以多行

> 也可以巢狀引用
>
> > 這是第二層引用
```

顯示效果：

> 這是引用文字
> 可以多行

> 也可以巢狀引用
>
> > 這是第二層引用

常用於強調重要提醒或引用外部來源。可配合 Emoji 使用，如：> ⚠️ 警告：...

- ✅ **用引用標註重要提醒或注意事項**
- ✅ **可以配合 emoji 提高辨識度**
  - 範例：`> ⚠️ **警告**：此操作無法復原`

表格設計（Tables）

```
| 欄位1 | 欄位2 | 欄位3 |
| ----- | ----- | ----- |
| 資料1 | 資料2 | 資料3 |
| 資料4 | 資料5 | 資料6 |
```

顯示效果：

欄位1	欄位2	欄位3
資料1	資料2	資料3
資料4	資料5	資料6

表格最適合用來呈現參數比較、規格表或步驟對比。

對齊方式

```
| 靠左 | 置中 | 靠右 |
| :--- | :--: | ---: |
| AAA  | BBB  |  CCC |
```

顯示效果：

靠左	置中	靠右
AAA	BBB	CCC

- ✅ **表格標題要清楚明確**
- ✅ **每一欄的資料類型要一致**
- ✅ **避免過度複雜的表格**（超過 5 欄建議拆分）
- ✅ **使用表格呈現結構化資料**（對比、規格、步驟等）

水平線

```
---
```

顯示效果：

- ✅ **統一使用** `---` （三個減號）
- ✅ **用水平線分隔不同的主題區塊**

第三章: AI 友善撰寫原則

提升 AI 理解率, 減少開發幻覺

原則一：杜絕模糊指稱

✅ **正確範例**：

```
# 專案文件

## 1. 系統架構

### 1.1 前端架構

- React 18
- TypeScript 5

### 1.2 後端架構

- Node.js
- Express

## 2. 安裝步驟

### 2.1 環境需求

...
```

❌ **錯誤範例**（層級混亂）：

```
# 專案文件

#### 系統架構

## 前端

### React

## 安裝
```

原則二：杜絕模糊指稱

❌ 模糊的寫法

這個功能很重要。

按那邊的按鈕。

✅ 精準的寫法

使用者登入功能 很重要。

點擊**畫面右上角的存檔按鈕** 。

```
這個功能很重要。
那個按鈕在右上角。
如上所述...
```

✅ **明確的寫法：**

```
**使用者登入功能**很重要。
**儲存按鈕**位於畫面右上角。
如**第 2.3 節**所述...
```

原則三：檔案路徑與名稱要用反引號

❌ **不清楚：**

```
請開啟 index.tsx 檔案
修改 src/app 資料夾下的檔案
```

✅ **清楚：**

```
請開啟 `index.tsx` 檔案
修改 `src/app/` 資料夾下的 `layout.tsx` 檔案
```

為什麼重要？

- AI 可以根據語言提供正確的語法建議
- 避免 AI 誤判程式語言
- 提高程式碼分析準確度

原則四：檔案路徑與名稱要用反引號

❌ **不清楚：**

```
請開啟 index.tsx 檔案
修改 src/app 資料夾下的檔案
```

✅ **清楚：**

```
請開啟 `index.tsx` 檔案
修改 `src/app/` 資料夾下的 `layout.tsx` 檔案
```

原則五：使用明確的執行性動詞

❌ **模糊：**

- 處理資料
- 做一些檢查
- 改一下設定

✅ **明確：**

- 驗證使用者輸入的 email 格式
- 檢查密碼是否符合 8 字元以上規則
- 修改 `config.json` 中的 API endpoint 設定

```
---

### 3.7 使用標記強調重要資訊

使用 emoji 或特殊符號讓 AI 辨識重要性：

```markdown
✅ 正確做法
❌ 錯誤做法
⚠️ 警告
💡 提示
🔍 注意
📝 補充說明
```
```

原則六：拒絕程式碼省略

避免在代碼區塊中使用 // ... 略。如果代碼太長，應將其拆分為多個小區塊並各別說明。

關鍵：AI如果看不到上下文，就無法發現邏輯錯誤

原則七：步驟的完整性

教學或安裝手冊必須包含：**前提環境 ➔ 具體動作 ➔ 預期結果**。

範例：執行 `node -v` ➔ 終端機應顯示版本號。若無顯示，請參閱錯誤排解。

原則八:結構化參數表格

當有一系列參數或配置時，表格能將 AI 的資料提取準確率提升

參數名稱	型別	說明
userId	String	不可重複的用戶識別碼

表格要有明確的標題和類型

❌ **標題不清楚：**

名稱	值
A	123
B	456

✅ **標題明確：**

設定項目	預設值	說明
port	3000	伺服器監聽埠號
timeout	5000	請求逾時時間（毫秒）

進階技巧

GitHub 風格的任務列表

```
- [x] 已完成的任務
- [ ] 未完成的任務
- [ ] 另一個未完成的任務
```

顯示效果：

- [x] 已完成的任務
- [ ] 未完成的任務
- [ ] 另一個未完成的任務

折疊區塊

```
<details>
<summary>點我展開詳細內容</summary>

這裡是折疊起來的內容。
可以放很多文字或程式碼。

</details>
```

顯示效果：

▶ 點我展開詳細內容

顯示效果：

▼ 點我展開詳細內容
這裡是折疊起來的內容。
可以放很多文字或程式碼。

第四章: Mermaid 流程圖語法

免開軟體, 用文字直接「畫出」邏輯

繪製流程圖的基本宣告

語法說明

A --> B: 一般連結

A -.-> 心B: 虛線連結

A -- 文字 --> B: 帶說明連結

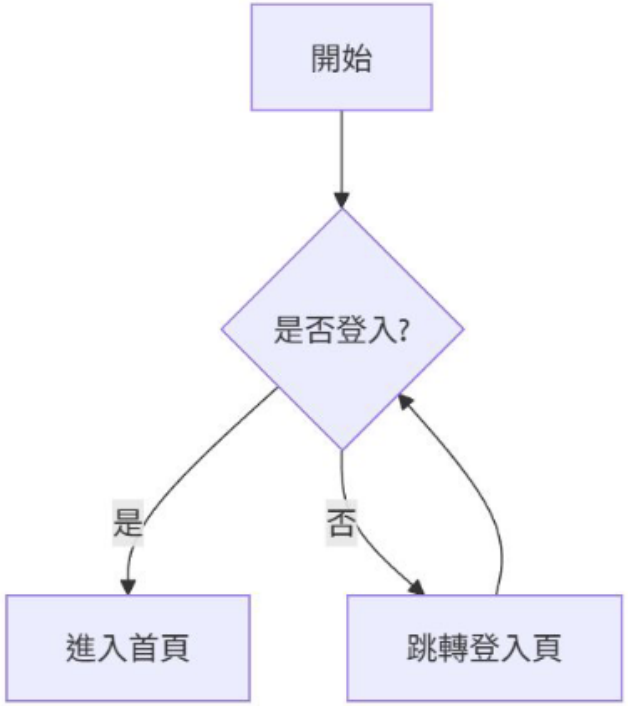
A ==> B: 粗線連結

語法說明：

- `graph TD`：表示 **T**op to **D**own (由上往下) 的流程圖。若要由左往右可用 `graph LR`。
- `-->`：箭頭連接線。
- `[文字]`：方形節點 (通常用於動作)。
- `{文字}`：菱形節點 (通常用於判斷)。
- `(文字)`：圓角節點 (通常用於開始/結束)。

常用形狀速查

形狀	語法	用途
方形	`[文字]`	處理步驟、動作
圓角	`(文字)`	開始、結束
菱形	`{文字}`	判斷 (Yes/No)
圓形	`((文字))`	連接點



使用 ```mermaid` 開頭的程式碼區塊：

```mermaid
graph TD;
    A[開始] --> B{是否登入?};
    B -- 是 --> C[進入首頁];
    B -- 否 --> D[跳轉登入頁];
    D --> B;
```

🤖 AI 友善提示

最強大的用法是**請 AI 幫您畫**！您可以這樣對 AI 說：

> 「請幫我用 Mermaid 畫一個『使用者註冊』的流程圖，包含檢查 Email 格式和密碼強度的邏輯。」

AI 就會自動產出對應的 Mermaid 程式碼，您只要複製貼上即可！

作 業 一

合併下周一起交

作業一：個人履歷 (每一項都必須符合指定的 Markdown 語法)

1. 文件結構 (10分)

[5分] 主標題：第一行必須是 `# A000 黃XX的履歷` (H1級別)

[5分] 副標題一：關於我 ，副標題二：我學過的程式課程 `##` (H2)

2. 關於我 (10分)

使用 ###(H3) X2 項，介紹自己，例如姓名、年級

[二擇一] 興趣無序列表：使用 `-` 或 `*` 符號列出項目 (例如：興趣、技能)

[二擇一] 興趣有序列表：使用 `1. 2. 3.` 符號列出順序 (例如：學經歷)

3. 程式碼展示 (10分)

[5 分] 程式碼區塊：必須使用三個反引號 (```) 包夾程式碼

[5 分] 語言標註：反引號後必須寫上語言名稱 (如 ```javascript)，讓程式碼有顏色

4. 表格製作 (20分)

表格語法：必須使用 `|` 和 `-` 製作至少兩欄的表格 (如技能熟練度)

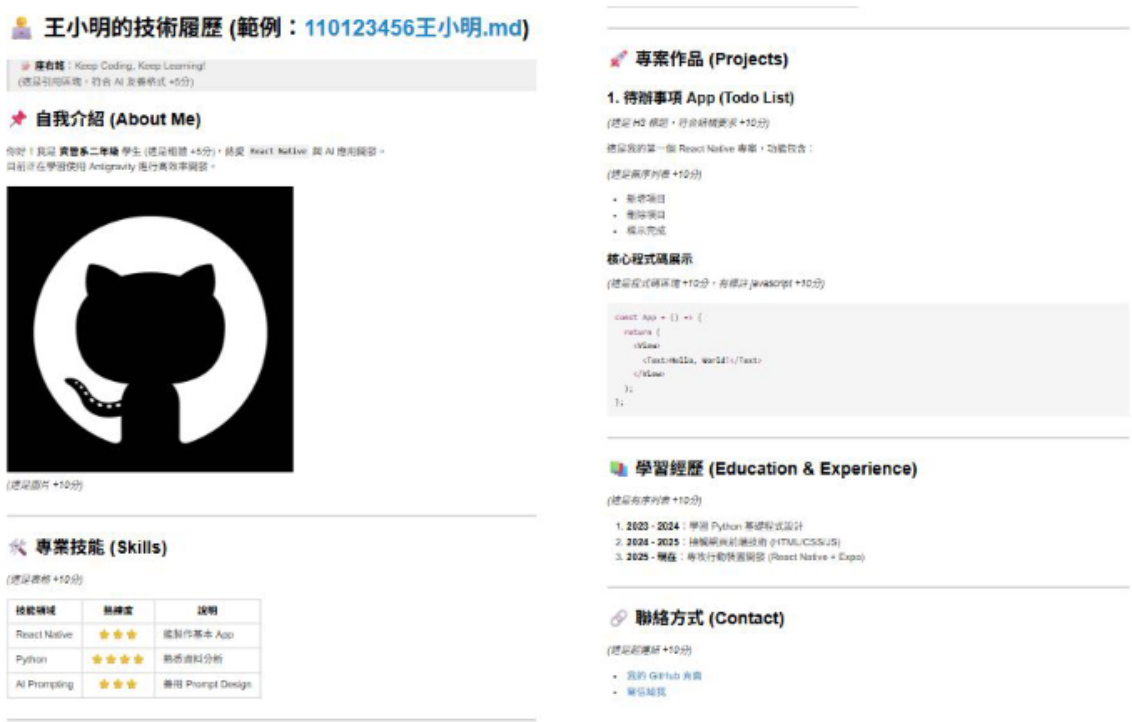
5. 流程圖繪製 (20分)

Mermaid：使用 `mermaid` 語法繪製一個簡單的流程圖 (如學習計畫或是技能樹)

6. AI 友善格式 (10分)

[5 分] 引用區塊：使用 `>` 標示自我介紹或座右銘

[5 分] 關鍵字強調：使用 `**粗體**` 或 `反引號` 強調重點關鍵字



Markdown 個人履歷製作（100分）

檔名：學號+姓名.md (例如：110123456王小明.md)

🎯 作業目標

- 📖 學會 Markdown 基本語法
- 建立清楚文件結構
- 練習自我介紹與資訊整理

📝 作業內容

1 文件結構 (15分)

- # A000 黃XX的履歷
- ## 關於我
- ## 我學過的程式課程
- 標題階層正確

2 關於我 (20分)

- 使用兩個 ### (基本資料 / 我的目標)
- 列出興趣或專長 (列表格式)

3 程式碼展示 (15分)

- 使用 ``` 包住程式碼
- 標註語言 (如 ```python)
- 至少一段可執行程式

4 表格 (20分)

- 製作技能表 (至少2欄3列)
- 使用 | 與 -

5 Mermaid 流程圖 (20分)

- 使用 ```mermaid
- 主題：學習計畫或成長路徑

6 AI友善格式 (10分)

- > 座右銘一句
- 使用 ** 粗體 ** 或 `反引號`
- 排版整齊

作業一：履歷範例 (每一項都必須符合指定的 Markdown 語法)

A000 王小明的履歷

關於我

基本資料

我叫王小明，目前就讀資訊工程學系大一。
來自台中，對**程式設計**與**人工智慧**非常有興趣。

我的目標

我希望未來能成為一名優秀的 `軟體工程師`，
並熟悉 **Python**、**Web 開發** 與資料分析技術。

我的興趣

- 打籃球
- 看科技新聞
- 研究新技術
- 製作小專案

> 每天進步一點點，就是成功的開始。

我學過的程式課程

目前正在學習：

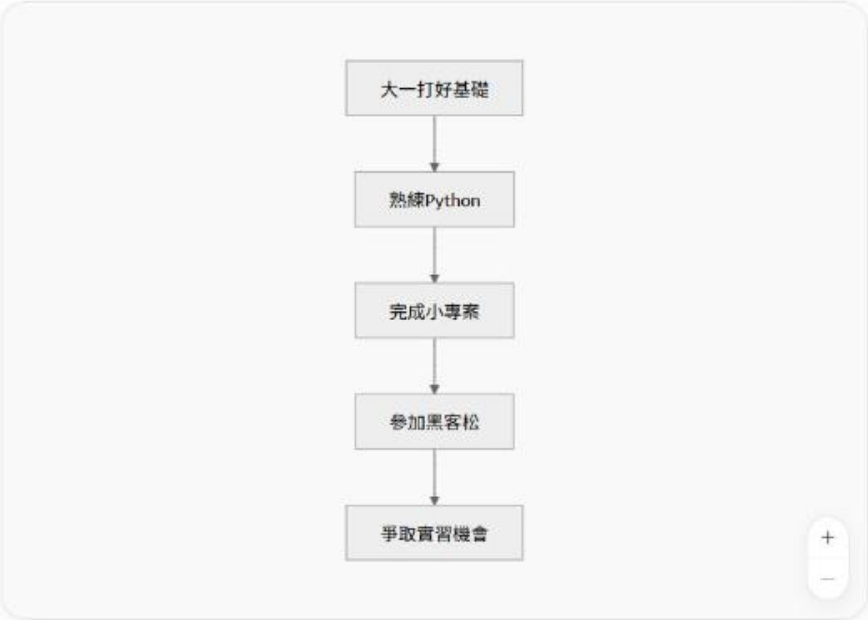
1. 計算機概論
2. 程式設計（Python）
3. 數位邏輯

未來想學：

- Web 前端開發
- App 開發
- 人工智慧基礎

技能表

技能
Python
HTML
Git
C語言

我的學習計畫（流程圖）

Mermaid